

Lesson 03: Terraform Installation

Learning Objectives

- Install Terraform
- Understand the Terraform CLI commands
- Learn the Terraform workflow: `init` → `plan` → `apply` → `destroy`
- Create your first Terraform configuration
- Understand state, desired state, and current state concepts

1. What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool by HashiCorp. It lets you define cloud resources in declarative configuration files and then provision them predictably.

Key concepts:

Concept	Meaning
Desired State	What you declare in <code>.tf</code> files — the infrastructure you want
Current State	What actually exists in the cloud provider
State File	Terraform's record of what it created, stored in <code>terraform.tfstate</code>
Plan	A diff between current state and desired state — shows what will change
Apply	Executes the plan to reach the desired state

2. Install Terraform

macOS

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Linux (Debian/Ubuntu)

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform
```

Windows

```
choco install terraform
```

Or download from: <https://developer.hashicorp.com/terraform/downloads>

Verify

```
terraform version
```

Expected output: **Terraform v1.x.x**

Enable tab completion (optional):

```
terraform -install-autocomplete
```

3. The Terraform CLI Commands

Command	Purpose
<code>terraform init</code>	Initialize a working directory, download providers
<code>terraform fmt</code>	Format configuration files to canonical style
<code>terraform validate</code>	Check configuration for syntax errors
<code>terraform plan</code>	Show what changes would be made
<code>terraform apply</code>	Execute the changes to create/update resources
<code>terraform destroy</code>	Delete all resources managed by the configuration
<code>terraform state list</code>	List resources in the state file
<code>terraform state show</code>	Show details of a specific resource in state
<code>terraform output</code>	Display output values
<code>terraform graph</code>	Generate a dependency graph (DOT format)

4. Your First Terraform Configuration

Create a new directory and file:

```
mkdir terraform-hello
cd terraform-hello
```

```
touch main.tf
```

Add this to `main.tf`:

```
resource "local_file" "hello" {  
  filename = "hello.txt"  
  content  = "Hello, Terraform!"  
}
```

This uses the `local` provider (bundled with Terraform) to create a text file.

Step 1: Initialize

```
terraform init
```

Output:

```
Initializing the backend...  
Initializing provider plugins...  
Terraform has been successfully initialized!
```

`init` prepares the working directory. For the `local` provider, no plugins need downloading.

Step 2: Format and Validate

```
terraform fmt  
terraform validate
```

`fmt` rewrites your files in canonical format. `validate` checks for errors.

Step 3: Plan

```
terraform plan
```

Output:

```
Terraform will perform the following actions:  
  
# local_file.hello will be created  
+ resource "local_file" "hello" {
```

```
+ content      = "Hello, Terraform!"
+ filename     = "hello.txt"
+ id           = (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

The **+** sign indicates a resource will be created. No resources exist yet.

Step 4: Apply

```
terraform apply
```

Terraform will prompt for confirmation. Type **yes**.

Output:

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Check the result:

```
cat hello.txt
```

Expected: **Hello, Terraform!**

Step 5: Inspect State

```
cat terraform.tfstate
```

You'll see a JSON representation of the created resource, including its **id**, **content**, and **filename**.

```
terraform state list
```

Output: **local_file.hello**

```
terraform state show local_file.hello
```

Shows detailed attributes of the resource.

Step 6: Destroy

```
terraform destroy
```

Type **yes** when prompted.

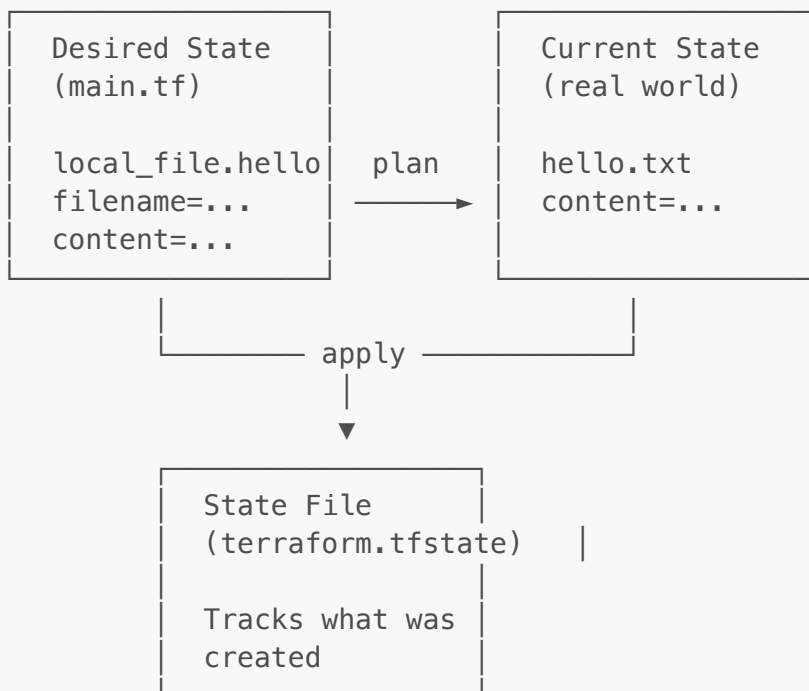
Output:

```
Destroy complete! Resources: 1 destroyed.
```

Verify the file is gone:

```
cat hello.txt
# Should show: cat: hello.txt: No such file or directory
```

5. Understanding State

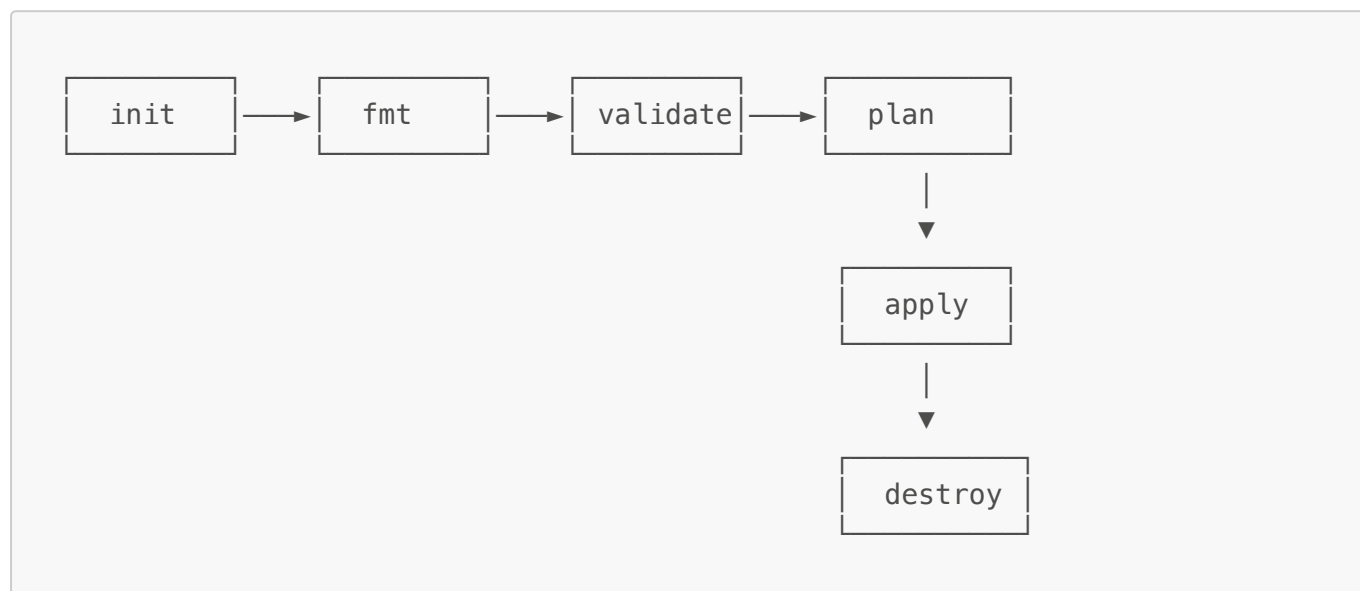


- **Desired State:** What you write in **.tf** files
- **Current State:** What actually exists (AWS resources, files, etc.)
- **State File:** Terraform's mapping between the two — it knows **local_file.hello** created **hello.txt**

When you run **terraform plan**, Terraform:

1. Reads desired state from `.tf` files
 2. Reads state file to see what was previously created
 3. Checks real world (for the `local` provider, it checks if the file exists)
 4. Computes the diff and shows it
-

6. Terraform Workflow Summary



This is the core loop you will repeat in every lesson going forward.

7. Key Takeaways

- Terraform uses declarative configuration — you describe the end state, not the steps
- `init` is always the first command in a new directory
- `plan` shows what will change without making changes (safe to run)
- `apply` executes changes and requires confirmation
- `destroy` tears everything down
- The state file (`terraform.tfstate`) is Terraform's source of truth for what exists
- The `local` provider is built-in — no download needed (unlike AWS provider, coming next lesson)

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer - Eyuel M.